

Deloppgave 2

Prompt

Sammenlign de tre ulike html dokumentene som er vedlagt og gjør en vurdering på hvilken løsning som er best. Målet med nettsiden er at brukeren skal skrive inn et ord i input-feltet, og når brukeren klikker på knappen skal nettsiden skrive ut følgende: «Ordet X består av Y tegn», hvor X er ordet brukeren skrev inn i input-feltet og Y er antallet tegn ordet består av.

Copilots vurdering:

Jeg har sammenlignet de tre løsningene: [copilot](#), [gemini](#) og [uiogpt](#). [\[copilot | HTML\]](#), [\[gemini | HTML\]](#), [\[uiogpt | HTML\]](#)

Vurderingskriterier

For oppgaven er det viktig å se på:

1. Om funksjonaliteten er korrekt.
2. Om koden er robust.
3. Om koden er lesbar og vedlikeholdbar.
4. Brukervennlighet.
5. HTML- og JavaScript-kvalitet.

1. Copilot-løsningen

Fordeler

- Oppfyller oppgaven korrekt. [\[copilot | HTML\]](#)
- Bruker moderne JavaScript (const og template literals). [\[copilot | HTML\]](#)
- Ren og lettest kode. [\[copilot | HTML\]](#)
- Ganske ryddig styling. [\[copilot | HTML\]](#)

Ulemper

- Ingen validering hvis brukeren klikker på knappen uten å skrive noe. Da vises:
- Ordet består av 0 tegn

[\[copilot | HTML\]](#)

Karakter

8/10

2. UiOGPT-løsningen

Fordeler

- Oppfyller oppgaven korrekt. [\[uiogpt | HTML\]](#)
- Har tilgjengelighetsforbedring ved å bruke <label> koblet til input-feltet. [\[uiogpt | HTML\]](#)
- Koden er enkel å forstå. [\[uiogpt | HTML\]](#)

Ulemper

- Bruker eldre JavaScript-stil med var i stedet for const eller let. [\[uiogpt | HTML\]](#)
- Strengsammensetting med + i stedet for template literals. [\[uiogpt | HTML\]](#)
- Tegnkodingen ser ut til å være feil i resultatstrengen (består i stedet for består). [\[uiogpt | HTML\]](#)
- Har ingen håndtering av tom input. [\[uiogpt | HTML\]](#)

Karakter

6/10

3. Gemini-løsningen

Fordeler

- Oppfyller oppgaven korrekt. [\[gemini | HTML\]](#)
- Har validering av tom input med:

JavaScript

1

if (ord.trim() === "")

Show more lines

[\[gemini | HTML\]](#)

- Gir en tydelig melding når brukeren ikke har skrevet inn noe. [\[gemini | HTML\]](#)
- Bruker moderne JavaScript (const og template literals). [\[gemini | HTML\]](#)
- Har ryddig struktur med egne variabler for input og resultatfelt. [\[gemini | HTML\]](#)
- Har den mest profesjonelle og moderne utformingen av brukergrensesnittet. [\[gemini | HTML\]](#)

Ulemper

- Mangler et eksplisitt <label>-element, noe som kunne forbedret tilgjengeligheten. [\[gemini | HTML\]](#)

Karakter

9/10

Rangering

Plass	Løsning	Vurdering
 1	Gemini	Best total løsning
 2	Copilot	Korrekt og ren løsning
 3	UiOGPT	Fungerer, men litt eldre og mindre robust

Konklusjon

Hvis jeg skulle velge den beste løsningen ut fra både funksjonalitet, kodekvalitet og brukeropplevelse, ville jeg valgt [gemini](#). Den er den eneste som håndterer tom input på en god måte, bruker moderne JavaScript og har den mest gjennomarbeidede brukeropplevelsen. [\[gemini | HTML\]](#)

Hvis tilgjengelighet hadde vært det absolutt viktigste kriteriet, kunne man argumentert for at [uiogpt](#) gjør én ting bedre ved å bruke `<label>`, men totalt sett er **Gemini-løsningen den sterkeste av de tre**. [\[gemini | HTML\]](#), [\[uiogpt | HTML\]](#)

Geminis vurdering:

Løsning 2 (Dokument 2) er den klart beste løsningen totalt sett.

Sammenligning av dokumentene

Kriterium	Dokument 1 HTML	Dokument 2 HTML	Dokument 3 HTML
Brukervennlighet & Validering	Ingen validering – viser tom streng dersom brukeren ikke har skrevet noe.	Inneholder tomrom-sjekk (<code>ord.trim() === ''</code>) som ber brukeren skrive inn et ord først.	Ingen validering – viser tom streng dersom feltet er tomt.
Responsivt Design & CSS	Bruker flex på body for å sentrere. Enkel styling uten hover-effekter på knappen.	Moderne og stilrent «kort-design» med god responsivitet (<code>max-width: 450px</code>), hovstil på knapp og fokus ramme.	Enkelt layout med fast margin: 40px. Mangler viewport-metatag for mobilskjermer.

Kriterium	Dokument 1 HTML	Dokument 2 HTML	Dokument 3 HTML
Kodekvalitet (JS)	Bruker modern ES6 (const, template literals). Oppfyller tekstkravet presist.	Ren og strukturert ES6-kode. Bruker const, template literals og har god layout-stabilitet (min-height).	Bruker eldre JS-syntaks (var) og streng-konkatinerer (+) istedenfor template literals.
Universell utforming (Accessibility)	Mangler kobling mellom tekst og felt.	Har god visuell kontekst, men mangler eksplisitt <label>.	Har eksplisitt <label> knyttet til feltet med for="ordInput".

Vurdering og konklusjon

- Dokument 2 er best: Det skiller seg ut ved å håndtere feilsituasjoner (tom input), har den mest profesjonelle visuelle utformingen med god brukeropplevelse (fokus- og hover-tilstander), og bruker moderne JavaScript-praksis.
- Dokument 1: Et helt ok og minimalt alternativ, men mangler validering.
- Dokument 3: Har det beste tilgjengelighetsgrepet ved å bruke <label>, men trekkes ned av utdatert JavaScript-syntaks (var) og manglende mobil-optimalisering (viewport).

En ideell løsning ville vært **Dokument 2**, men supplert med en <label for="ordInput"> fra Dokument 3 for optimal tilgjengelighet.

Kommentar fra Téa: Dokument 1 = Copilot, Dokument 2 = Gemini, Dokument 3 = UiO-GPT

UiO GPTs vurdering

Alle tre løsningene gjør i hovedsak det de skal, men én av dem skiller seg tydelig ut som best helhetlig løsning.

1. Løsning med validering og "kort"-design (UiO GPT-filen)

Disse delene hører sammen:

- Layout og styling med `.card`, fokus-styling på input, osv.
1 og 2
- JavaScript med validering og utskrift av setningen:
3 og 4

Egenskaper:

- **Oppfyller kravet til teksten nøyaktig:**
`resultatFelt.textContent = \Ordet ordbestår av {antallTegn} tegn`;`
4
- **Håndterer tomt input-felt:**
Sjekker if (`ord.trim() === ""`) og viser meldingen
Vennligst skriv inn et ord først.
3
- **God brukeropplevelse:**
 - Pen, sentrert layout med “card” og skygge. 1
 - Min-høyde på resultatfeltet (`min-height: 24px;`) gjør at siden ikke “hopper” når teksten vises.
3
 - Fokus-styling på input-feltet. 2
- **God kodekvalitet:**
 - Bruker `const` og ryddig DOM-tilgang. 3
 - Funksjonelt skilt ut i function `tellTegn() { ... }`.

Vurdering:

Dette er den beste løsningen totalt sett: den er korrekt, robust (håndterer tom input), gir god tilbakemelding til brukeren, og koden er moderne og ryddig.

2. Løsningen med enkel, sentrert layout (Copilot-filen)

Her har du:

- CSS med sentrert layout (`flex`, `min-height: 100vh` osv.). 5
- JavaScript-funksjon som teller tegn og skriver ut riktig tekst:
6

Egenskaper:

- **Oppfyller hovedkravet:**
Ordet `{ord}` består av `{antallTegn}` tegn
6
- **Enkel og oversiktlig kode**, men:
 - Ingen håndtering av tomt felt; brukeren får da en setning med tomt ord.

- Ingen ekstra brukerhjelp eller feilmeldinger.

Vurdering:

Teknisk korrekt og enkel, men mindre brukervennlig enn den første løsningen fordi tom input ikke håndteres.

3. Løsningen med enkel margin-layout (Gemini-filen)

Her er det:

- Klassisk layout med margin og enkel styling. 7
- JavaScript som bruker strengkonkatenering:
resultatDiv.textContent = "Ordet " + ord + " består av " + antallTegn + " tegn";
som ligger delt mellom segmentene:
8 og 9

Egenskaper:

- **Oppfyller kravet til teksten** (selv om består er delt i to segmenter):
"Ordet " + ord + " består av " + antallTegn + " tegn"
8 + 9
- **Enkel kode**, men:
 - Ingen behandling av tomt input.
 - Litt mer "gammeldags" JavaScript (var og strengkonkatenering i stedet for template literals).

Vurdering:

Fungerer helt OK og er enkel å forstå, men svakere både på moderne praksis og brukervennlighet enn den første løsningen.

Konklusjon

- **Best løsning:** den første (UiO GPT) med "kort"-layout og validering (1, 3, 4)
 - Oppfyller kravene helt nøyaktig.
 - Har god tilbakemelding ved tomt input.
 - Har gjennomført og brukervennlig design.
 - Bruker moderne og ryddig JavaScript.

De to andre løsningene er teknisk korrekte, men den første skårer klart høyest på både robusthet og brukeropplevelse.

Deloppgave 3

Er KI-modellene enige?

Copilot og Gemini var stort sett enige. Begge mente at Gemini gjorde den beste jobben og poengterte at det var den eneste som hadde validering for når man ikke skriver noe i input-feltet, og at Gemini har det mest profesjonelle brukergrensesnittet. De var enig i at UiO-GPT hadde utdatert JavaScript-syntaks, men ga pluss til UiO-GPT for at den hadde beste tilgjengelighetsgrep gjennom bruken av <label>-tagen.

UiO-GPT var derimot uenig med de to andre. Den kjente ikke igjen at sin egen kode brukte utdatert javascript-syntaks, men mente at Gemini sin var utdatert. Den hallusinerte at den ga en validerings melding ved tom input-boks, selv om det kun var Gemini som gjorde det, og skrev at dette trakk Copilot sin kode ned. Den mente at sin egen kode var den beste.

Hvilken KI-modell gir, etter din mening, den beste / mest riktige svaret?

I første del av oppgaven er jeg enig med Gemini og Copilot at Gemini har den beste løsningen. Etter min mening er Gemini den sterkeste grafisk sett, og har økt brukervennlighet gjennom valideringsmeldingen. Visuelt mener jeg at Copilot er svakest gjennom at input-feltet og knappen ikke har noe luft imellom seg, og mangel på padding gir et uprofesjonelt og uferdig inntrykk. I koden står det at det skal være padding på knappen og input-feltet, men for meg kan det virke som at det er noe feil med hvordan CSS-syntaksen er skrevet.

I andre del av oppgaven mener jeg at Copilot og Gemini står ganske likt, siden de ga mye av den samme informasjon. Jeg foretrekker måten Copilot presenterte informasjonen sin på fordi den tok for seg en og en fil og ga fordeler og ulemper på hver og en fil. Gemini sitt sammenligningsskjema var ganske fin å ha som supplement til Copilot, men skulle ønske den skulle ønske den utdypet hver enkelt mer.

Copilot poengterte derimot en feil i UiO-GPT som jeg er usikker på stemmer eller ikke. «Tegnkodingen ser ut til å være feil i resultatstrengen (består i stedet for består)». Jeg kan ikke se denne feilen i UiO-GPT filen eller på nettsiden. Verken Gemini eller UiO-GPT poengterte denne feilen, og sammen med at jeg heller ikke ser noe som er galt så tror jeg at feilen ligger på Copilot sin side.

UiO-GPT skal ha for at den er den eneste modellen som viste til hvilken kodelinje den refererte til i svaret sitt. Jeg synes derimot at det er vanskelig å stole på at den sier stemmer med tanke på at den hallusinerte at dens egen kode hadde funksjoner den ikke hadde, den var helt uenig med de andre modellene og har ifølge de andre modellene utdatert JavaScript-syntaks.

Hvor godt er du i stand til å vurdere hvilken av KI-modellene som gir det beste / mest riktige svaret?

Jeg kjenner igjen mye av koden fra emnet webutvikling, og jeg har generelt en grei forståelse av tilbakemeldingene som KI-modellene har gitt. Jeg kjenner meg fortsatt ikke trygg nok på HTML eller CSS til å gi en god vurdering av hva koden burde gjøre annerledes. Eksempel er hvordan CSS-koden til Copilot som skal gi padding ikke virker til å gå igjennom. Ingenting virker til å skje når jeg endrer på verdiene, men jeg har ikke nok erfaring med inline kode til å vite om den er skrevet riktig eller ikke, om det er en annen grunn til at det ikke funker, eller om den rett og slett skal være sånn som den står.

Jeg føler meg ganske trygg i min vurdering av at UiO-GPT er vanskelig å stole på med oppgaven med tanke på hallusinasjon, men vurderingen om at JavaScript-Syntaksen er utdatert er kun basert på at de to andre modellene mente det. Det er tydelig at man bør dobbeltsjekke andre kilder før man står bak det KI sier, spesielt når modellene er så uenig med hverandre.

Hvis du skulle løst denne oppgaven igjen, ville du promptet mer eller annerledes?

- Jeg ville ha bedt Gemini og Copilot om å ha med hvilken kodelinje de refererte til.
- Jeg ville ha sendt de tre ulike sammenligningene tilbake til alle modellene og funnet ut hvordan de vurderer de andre sammenligningene.
- Jeg ville ha spurt hvorfor Copilot sin padding ikke virker, og hva man måtte endret for å få det til å funke.
- Jeg ville ha poengtert til UiO-GPT at den hadde hallusinert sin egen verifisering, og at koden var utdatert for å se om den ville gitt et sterkere svar.